

# Intern: likely defense questions

Suggested answers use results computed from the logs on October 07, 2026. Re-run this script before the defense to refresh them.

**How to use this.** Questions marked ★ are the ones most likely to come up; rehearse those out loud first. Each answer is a starting point; say it in your own words.

The shaded **Watch out** notes flag questions that are traps, or where overclaiming would cost you.

## Learning signal (raised by the instructor; answer this first)

---

### ★ 1. What is the learning signal? What exactly does Intern learn from?

The user's own demonstrated next action. Every recorded step becomes one training example: the screen as it was (every element and its state) paired with what the user did next (which element they clicked, whether they clicked or typed, and which source value they typed). It is supervised imitation: the 'correct answer' is always what this user actually did. There is no reward and no hand-written label.

**Watch out:** Say 'supervised, from demonstrations, no reward' in the first sentence. That is the direct answer to the question.

### ★ 2. Is there any learning signal after training, when Intern is actually running?

Per scope. Scope #3: yes, online. Every time the user accepts or overrides a decision, the sender history is updated immediately and the example is stored for the next retraining of the habits model. Scope #1: not yet. The correction hook (Dagger: the user fixes a mistake and that fix becomes training data) exists but captured no steps in live runs, so today the model only improves when it is retrained on new recordings. Scope #2: the pass/fail rule is induced from the demonstration, so a new demonstration changes it.

**Watch out:** Don't say Scope #1 learns from its mistakes while running. It doesn't yet.

### ★ 3. Behavioral cloning only sees correct actions. Where is the negative signal?

There is none in pure cloning, and that is its known weakness: the model never sees what to do after its own mistake, so errors compound on long tasks (the Scope #1 failure). Negative signal would come from corrections (Dagger) or from a reward (reinforcement learning), both future work. In Scope #3, an override is a negative signal: it records that Intern's suggestion was wrong.

### ★ 4. How clean is the signal? What if a demonstration is messy or wrong?

Noise is removed before training: clicks that select a value in an open dropdown (which land on the field underneath), clicks outside the form window, clicks on empty panes and repeated duplicates. A step whose label cannot be determined (for example, a typed value that matches no source element) is marked 'ignore' so it adds no loss at all, rather than teaching something wrong.

**Watch out:** Cleaning must not remove real signal. One early fix wrongly discarded lone Tab presses, which are legitimate navigation the user really did. It was caught by testing and reverted.

## 5. Can the labels contradict each other?

Yes, and that was measured. The pointer predicts an element by its position in the list. If the same position means 'First Name' in one recording and 'State' in another (because the order the accessibility tree reports elements in shifts), the labels contradict each other and no model can learn them. A diagnostic script (`diagnose_click_labels.py`) counts how many clicks land inside an element at all and how many distinct fields each position stands for.

## 6. Is the signal strong enough for rare steps?

Rare steps carry a weak signal: switching tabs or pressing Submit happens once per form among dozens of field clicks. Two remedies are used: clicks on rarely-clicked fields are weighted up in the loss (by inverse frequency, detected automatically), and the 'form complete, click Submit' step is oversampled. This is still the weakest part of the signal and the main reason click accuracy is 52.2% (n=6,401).

## 7. Does the signal teach this user's habits, or just the form's layout?

The clone test answers this: the same architecture trained on top-down demonstrations filled top-down, and trained on bottom-up demonstrations filled bottom-up (June 2026, earlier version). The layout was identical in both, so what it learned was the user's order.

# The idea and why it matters

---

## ★ 8. In one sentence, what is Intern?

A desktop agent that watches how a person does a repetitive computer task, learns their way of doing it, and then does it for them by looking at the screen and using the mouse and keyboard, like a person would.

**Watch out:** Keep it to one sentence. Panels test whether you can explain it simply.

## ★ 9. How is this different from RPA tools like UiPath or Power Automate?

RPA has to be scripted by hand by a developer, step by step, and breaks when the screen changes. Intern learns the task from demonstrations instead of a script, and it learns this user's own way of doing it.

**Watch out:** Don't claim Intern beats RPA. The RPA comparison (Objectives 11 and 12) has not been measured yet. Say the comparison tool is built and the data is still to be collected.

## ★ 10. How is it different from AI computer-use agents (Claude computer use, OpenAI Operator)?

Those agents do a task their own way, from a written instruction, usually with a large cloud model. Intern copies how this particular user does it (behavioral cloning), runs small models locally, and only asks a language model for the parts its learned habits can't settle.

**Watch out:** If asked 'why not just use one of them?', the honest answer is personalization and running locally, not that Intern is more capable today.

## 11. Who would actually use this?

Office staff who re-type the same kind of data every day: insurance intake forms, copying grades from a spreadsheet into a school portal, and sorting a busy inbox. Those are the three scopes.

## How it works (Methods)

---

### ★ 12. What exactly does the Transformer learn, and what does the LLM do?

The Transformer learns WHERE to act and in what order: which field or button comes next, the way the user did it. The LLM supplies WHAT value to type when a value has to be read from the source. Mechanical HOW (type into a text box, open a dropdown) follows the widget type.

**Watch out:** Expect 'so the LLM does the real work?' Answer: the order and the workflow, which is the personal part, are learned; the LLM only reads values.

### 13. Why behavioral cloning and not reinforcement learning?

Cloning learns directly from a few demonstrations of the real task, which is what a user can give. Reinforcement learning needs a reward and many thousands of trial runs on the live screen, which is slow and risky for real applications.

**Watch out:** Admit the known weakness of cloning: small mistakes pile up over long tasks. That is exactly why Scope #1's long form fails (see Results).

### ★ 14. How does Intern 'see' the screen?

Through Windows UI Automation, the same accessibility information screen readers use: every element's type, label, value and position. Perception is a swappable adapter, so a vision-based reader could replace it without touching the learning part.

**Watch out:** Objective 1 (vision-based perception, 95% detection) was not evaluated. Say so plainly: the system uses the accessibility tree, and vision is future work.

### 15. Did you hardcode anything for the specific tasks?

No field names, form layouts or task steps are written in the agent. Each scope passes its own small configuration. There are about five universal widget rules (for example, a dropdown is opened before choosing), which apply to any application.

**Watch out:** Be precise: 'no task-specific rules' is true; 'zero rules at all' is not.

### 16. How do you measure your objectives?

One script (objective\_metrics.py) computes every result straight from the run logs. An objective counts as met only if every scope that was measured met it. 'Not evaluated' means there is no data, which is different from 'not met'.

## Results

---

### ★ 17. How many of your 12 objectives did you meet?

2 met, 2 partially met, 4 not met, and 4 not evaluated (Objectives 1, 7, 11, 12). Every number comes straight from the logs; none were adjusted.

**Watch out:** Don't get defensive. Lead with what works, then say what doesn't and why.

★ **18. Your prediction accuracy (validation click accuracy) is 52.2% (n=6,401) against a 90% target. Why so low?**

The model was trained on a limited number of demonstrations of a long form with eight tabs, where tab switches are rare events. Rare steps are exactly what cloning learns worst. The plan is a larger recording campaign and retraining.

**Watch out:** Don't blame the data without a plan. Say what more data would fix and what it wouldn't.

★ **19. Scope #1 completes 0.0% (n=197) of runs end to end. Does it work at all?**

The pieces work: in a June 2026 clone test (test\_clone.py, earlier version), a model trained on top-down demonstrations filled top-down (74% exact) and one trained on bottom-up filled bottom-up (93%), proving it copies the user's order. What fails is finishing the whole 176-field form: errors pile up across tabs.

**Watch out:** This is the hardest question. Show the clone-test evidence, then own the end-to-end failure.

★ **20. Scope #2 completes 100.0% (n=210). Isn't that too good to be true?**

Completion there means every row was processed and verified without a crash. It does not mean every field was filled: some fields (for example 'Course') are left empty when the system isn't sure, because an empty field is safer than a wrong one.

**Watch out:** Don't let '100%' stand unqualified. A panel member who opens the output will see empty fields.

★ **21. Adaptability is 75.0% (n=24) on Scope #2. Is that meaningful?**

It lands exactly on the 75% threshold with a small sample (3 fields across 8 portal variants), and only 3 of 8 variants were fully resolved. It is a pass, but a weak one; Scope #3 measured 83.6% (n=292).

**Watch out:** Say 'weak pass' before they do.

**22. Scope #3 completes 51.7% (n=292) and hands 46.6% (n=292) to a person. Why?**

By design. When neither the user's habits nor the reasoning step is sure, the email is left for a person instead of guessed. A wrong reply sent in the user's name costs more than one the user handles.

**Watch out:** Have one example ready of an email it correctly refused to guess on.

**23. Objective 2 (encoding ambiguity) is 10.9% (n=19) against under 5%. What does that mean?**

About one in ten recorded screen states looked identical to another state but needed a different action, so the model couldn't tell them apart. Adding which fields are filled and which has focus already cut this; more state detail would cut it further.

★ **24. Why are Objectives 1, 7, 11, 12 not evaluated?**

Vision perception (1) is not built. Scalability (7) needs training at several data sizes, after the recording campaign. The RPA comparison (11, 12) needs timed runs of a traditional RPA tool on the same task; the comparison and significance-test tool is built and tested, but the runs have not been done.

**Watch out:** Never fill these gaps with estimates during the defense.

**25. Should we accept a thesis that misses most of its numeric targets?**

The contribution is a working, personalized, demonstration-learned agent across three different kinds of work, with an honest measurement pipeline that reports what the logs say. The misses are explained and each has a concrete cause and next step.

**Watch out:** This is for your adviser to help frame. Agree the line before the defense.

## The three scopes in detail

---

### 26. Scope #3: how does it decide to reply, forward, schedule, or leave an email alone?

First from the user's habits: a model trained on how they handled similar emails, then their history with that sender. Only what habits can't settle goes to a local LLM. The demo can be run in Habits only, Habits + reasoning, or Reasoning only.

### ★ 27. Does it send emails on my behalf?

No. It only creates drafts and calendar events. The email interface in the code has no send function at all, so it cannot send even by mistake.

**Watch out:** This answer reassures the panel. Say it with confidence.

### 28. What if the LLM makes up a meeting time?

It can't: dates and times are read with fixed rules from the email's own words, and only when the email states both a date and a clock time. Otherwise the email waits for a person. Checks like 'Am I free next Friday?' answer 'unclear' rather than guess.

### 29. Scope #2: how does it know which spreadsheet column goes into which form field?

It reads each field's label and looks for it among the column names, like a person would. Fields the names can't settle go to a local LLM, which is allowed to answer 'none'. Pass/fail is derived by a rule learned from a demonstration, not configured.

**Watch out:** The 'Course' field is currently left empty. Say it's a known gap with a fix in progress.

### 30. Scope #1: why does it struggle with the long form when short ones work?

Each step depends on the previous one, so one wrong click shifts everything after it. On a 176-field, 8-tab form those small errors add up, and the first click from a blank screen is the hardest. Letting the user correct mistakes during runs (Dagger) is the standard fix and is the next step.

## Generalization, speed, and data

---

### ★ 31. Would it work on an application it has never seen?

The agent itself has no application-specific code, and the screen reader plugs in per app. A new application still needs demonstrations of the new task. That is the point: it learns from the user, not from a developer's script.

**Watch out:** Don't claim zero-shot generalization. Scope #2's eight portal variants are the evidence you have.

### 32. Is it faster than doing the task by hand?

Not measured against a person yet. That is part of the RPA comparison still to be run. In fast mode, Scope #1 writes known values directly into fields instead of typing them.

**Watch out:** Don't guess a speed-up figure.

### 33. How many demonstrations does a new task need?

For one 10-field section, about 20 to 30 passes were needed. Whether the full form scales on data alone is an open question, and it's exactly what Objective 7 (scalability) will measure.

### **34. What happens if the window moves or the screen resolution changes?**

Clicks currently use screen positions read from the accessibility tree at that moment, so they follow the window. A change mid-step can still miss. This is listed as a known risk.

## **Privacy, safety, and ethics**

---

### **★ 35. Where does the recorded data go? Is it private?**

Everything stays on the user's machine. Demonstrations are stored locally, the language model runs locally in LM Studio, and the demo uses a practice mailbox, not a real account.

### **36. What stops it from doing something irreversible?**

It runs as a dry run unless told otherwise, creates drafts instead of sending, verifies every value it writes by reading it back, refuses to guess when unsure, and the user can stop it at any time.

### **37. Could someone use this to impersonate a user?**

It acts only on the user's own machine with their own access, the same as the user sitting there. Sending stays with the user because drafts are never sent automatically.

## **Validity of your method**

---

### **★ 38. Your sample sizes are small. Are the results statistically meaningful?**

Several are small (for example  $n=19$  for the ambiguity measure), and that is stated next to every number. Statistical-significance testing is Objective 12, which is not evaluated yet.

**Watch out:** Say the  $n$  out loud whenever you quote a percentage.

### **39. You built the test applications yourselves. Isn't that grading your own homework?**

Yes, that is a real threat to validity. The mitigation is that the Scope #2 portal comes in eight variants that each change one thing, and the agent has no code specific to any of them.

### **★ 40. Did you use AI tools to build this?**

Answer honestly and specifically: which tools were used, for what (for example, coding assistance), and what the team decided and verified itself, such as the design decisions, the experiments, and the tests.

**Watch out:** Agree on this answer as a team before the defense. Inconsistent answers here are worse than any single answer.

## **Looking ahead**

---

### **41. What is the biggest limitation?**

Long tasks: the cloned policy loses its place on long forms. Everything else (perception adapter, value reading, safety, measurement) is in place around it.

### **42. What would you do with six more months?**

A larger recording campaign and retraining; letting users correct mistakes during runs; the RPA comparison with significance tests; a vision-based perception adapter.

### 43. What did you learn?

Prepare a personal answer: for example, that honest measurement changed the conclusion more than any single model improvement did.

**Watch out:** Make this personal and short.

## Technical: the model

---

### ★ 44. Walk us through the Transformer's architecture.

Each on-screen element becomes one vector: 11 structured features (real or padding, bounding box, confidence, which window, focused, control type, filled) plus a 384-dimensional text embedding of its label from all-MiniLM-L6-v2. A linear layer with LayerNorm projects each element to 128 dimensions. The elements of one screen are mean-pooled (ignoring padding) into a single token. The last 4 screens and the actions between them are interleaved into a sequence with learned position embeddings, then passed through a 4-layer causal Transformer encoder (4 heads, feed-forward 256, dropout 0.1). The final token cross-attends back to the current screen's elements, and separate heads predict the action type, which element to click (a pointer), which source element holds the value, hotkeys and scrolling.

**Watch out:** Know the numbers cold: 128-d, 4 layers, 4 heads, 4 steps of history, up to 128 elements.

### ★ 45. Why a pointer head instead of predicting a class like 'click field 7'?

The number and order of elements change from screen to screen, so a fixed list of classes would not mean the same thing twice. A pointer scores every element on the current screen (a query from the sequence against a key per element) and picks the best one, so it works for any number of elements.

### 46. Why LayerNorm on the pointer head's query and key?

The pointer score is a dot product of two learned vectors. Without normalization their size grew during training and the scores diverged. Normalizing both keeps the scores in a stable range.

### 47. Why mean-pool each screen and then cross-attend, instead of feeding every element in?

Pooling keeps the sequence short and fast (one token per screen). But a pooled token can't point at a specific element, so the final token cross-attends to the current screen's elements. That is what lets the pointer pick a specific field.

### 48. Why causal masking?

The model predicts the next action, so each position may only see what came before it, the same as at run time. Otherwise training would peek at the answer.

### 49. Why such a small model?

There are only thousands of training samples, and every step must be fast on a normal laptop. A larger model would overfit this data and slow every step. More model size also did not fix the hardest failure, the very first click from a blank screen.

### 50. How do you handle a different number of elements on each screen?

Every screen is padded to 128 elements. Feature 0 is an 'is this real' flag that becomes a mask, so padding is ignored when pooling and can never be picked by the pointer.

### 51. Why all-MiniLM-L6-v2 for text?

It is small and fast, runs locally, and gives a 384-dimensional embedding where similar labels ('DOB', 'Date of Birth') land close together, so the model copes with different wording.

## Technical: training and data

---

### ★ 52. What is your loss function, and how do you handle class imbalance?

Cross-entropy on each head. Rare targets are up-weighted by the inverse frequency of the specific field clicked, normalized so the average weight is about 1 and capped to avoid blow-ups. Rarity is detected automatically, so no field name is hardcoded. The rare 'form complete, click Submit' step was also oversampled in the data.

### ★ 53. Is your validation set really separate from training?

Know your answer before the defense: whether the split is by individual step or by whole demonstration session.

**Watch out:** Classic trap. A step-level split puts near-identical neighboring screens in both sets and inflates accuracy. If your split is by step, say so and call it a limitation.

### ★ 54. Behavioral cloning suffers from compounding error. What did you do about it?

That is exactly the Scope #1 long-form failure. The standard fix is DAgger: run the policy, have the user correct its mistakes, add those corrections to the training data, retrain. A correction hook exists, but it captured no steps in live runs, so it is the next thing to fix.

**Watch out:** Don't claim DAgger is implemented. It is designed, not working.

### 55. Your accuracy metric is validation click accuracy (52.2% (n=6,401)). Why that one?

Which element to click is the personal part of the workflow (the order and the navigation), measured on held-out data. End-to-end completion is reported separately (Objective 8), because a policy can be locally accurate and still fail a long task.

**Watch out:** Its 95% range is 51%-53%: with that many samples the uncertainty is in the method, not the sample size.

### 56. How did you stop test runs from polluting your reported metrics?

The training log is shared with the test suite, which writes tiny synthetic runs. Those are filtered out explicitly before any number is reported, and a test checks it: a 4-sample test run logging 99% would otherwise have become the headline.

## Technical: perception and execution

---

### 57. What happens with apps that have no accessibility tree (games, canvases, remote desktops)?

UI Automation returns nothing useful there. There is an OCR fallback for text, but real support needs the vision adapter (Objective 1), which is future work. The adapter seam is built: every perception source must produce the same element schema, and the agent rejects one that doesn't.

### 58. How do actions actually happen on screen?

Mouse and keyboard through pyautogui by default. For speed, known text values can be written straight into a field with a Windows message (WM\_SETTEXT), and buttons can be pressed through accessibility 'invoke' or a Windows click message, falling back to a real click. Every written value is read back and checked.

**Watch out:** Expect: 'isn't writing directly into fields cheating the human-only rule?' Answer: it is a speed mode you can switch off; Transformer mode reaches every field by real clicks.



## 59. What do 'encoding ambiguity' (10.9% (n=19)) and 'transition mapping' (81.7% (n=19)) measure?

Ambiguity: the share of recorded screens whose encoding is identical to another screen that needed a different action, so the model literally cannot tell them apart. Transition mapping: the share of recorded interactions that could be tied to one specific element.

**Watch out:** Both rest on only 19 audited sessions; the 95% ranges are 3%-32% and 60%-93%.

## Technical: Scope #2 (sheet to portal)

---

### ★ 60. How does the column-to-field matcher work?

Each (column, field) pair gets 17 features: semantic similarity of the field's label, name and placeholder to the column header (MiniLM); lexical overlap (edit distance, Jaccard, containment, abbreviations); whether the column's values fit the field's type, pattern, range and length; structural fit; position; and option overlap for dropdowns. A one-to-one assignment is solved with the Hungarian algorithm, padded so a column can be assigned to nothing.

**Watch out:** Be precise about what runs in the demo: the live path matches labels first and asks the local LLM about the rest, because loading the embedding model added about 9 seconds per run. The 17-feature matcher is kept and measured in the evaluation.

### ★ 61. How does it learn the pass/fail rule instead of being told?

From the demonstration it finds which numeric column drives Remarks, which direction passes (higher or lower), and the interval the cutoff must lie in (the demonstrations narrowed it to between 74 and 85). It snaps the cutoff to the most plausible round value in that interval: a multiple of 5 first, then a whole number, then the midpoint. That gives 75. On the 1.00-5.00 portal the direction flips.

### 62. Why let the system leave fields empty?

Some columns are deliberate decoys. A wrong value in a grade portal is worse than an empty one a person fills in. The LLM may only answer with an exact column name or 'NONE', and if two fields claim the same column, one tie-break question is asked from the column's side.

### 63. Your LLM runs at temperature 0. Are its answers deterministic?

Not fully. Asking the same questions repeatedly through LM Studio gave different answers on some runs. That is one reason every LLM answer is checked against strict rules rather than trusted.

## Technical: Scope #3 (inbox)

---

### ★ 64. What is the 'habits' model in Scope #3?

A deliberately small network: one hidden layer of 16 units (ReLU, dropout 0.2). Its inputs are how similar the email's text embedding is to the average of the user's past emails for each decision (reply, forward, schedule, leave alone), plus the sender's history ratios. It only acts when at least 75% confident; otherwise the next layer decides.

**Watch out:** Why so small: one user gives few examples, and comparing to per-decision averages makes it data-efficient.

### **65. Where does the sender history come from?**

It is built automatically from the Sent folder, matched against recent inbox threads (who you replied to, who you forwarded), so no manual labelling is needed. It is updated every time the user accepts or changes a decision.

### **66. How is a meeting time extracted, and why not with the LLM?**

With fixed patterns that only fire when a calendar date and a clock time appear next to each other in the email. The year comes from when the email arrived. An LLM asked 'when is this?' always produces an answer, even when the email gives none, and a wrong calendar entry is worse than none.

### **67. How do you know Intern's decision was right, if the user just clicks the action they want?**

Each action is recorded as either confirming Intern's suggestion or overriding it. That difference feeds the accuracy metrics and the sender-history updates.

## **Technical: evaluation and statistics**

---

### **★ 68. What are the confidence intervals on your key results? For example, adaptability 75.0% (n=24).**

Using the Wilson interval, which behaves well at small samples: adaptability's 95% range is 55%-88%, and Scope #3 completion's is 46%-57%. So the adaptability 'pass' is consistent with anything from clearly failing to clearly passing.

**Watch out:** Offer the interval before they ask. It shows you understand your own numbers.

### **69. Which statistical test will you use against RPA, and why?**

Welch's t-test at the 0.05 level, per metric. Welch's version doesn't assume both tools have the same spread of results, which is unlikely for a learning agent compared with a fixed script.

**Watch out:** Be ready for 'what about multiple comparisons?': testing four metrics at 0.05 raises the chance of one false positive. A Holm or Bonferroni correction is the fix.

### **70. Why is an objective MET only if every measured scope met it, rather than averaging?**

An average lets one strong scope hide two failing ones. The first version did exactly that and reported end-to-end completion as met off Scope #2 alone. Mixed results are now reported as 'partially met'.

## **Technical: systems and engineering**

---

### **71. Why a local LLM (Qwen 2.5 7B in LM Studio) instead of a stronger cloud model?**

Privacy (emails and form data never leave the machine), no cost per call, and it works offline. The price is weaker reasoning, which is why every LLM answer is constrained and checked.

### **72. How does the agent work inside a window the user already opened?**

Launch starts Chromium with a remote-debugging port; Play connects to it over the Chrome DevTools Protocol and works in that tab. If none is open, Play opens its own. On this Chromium version a page opened from the command line made the connection hang, so the page is opened as a new tab after the browser starts.

### 73. How many tests are there, and what do they not cover?

Hundreds of unit, integration and end-to-end tests, including real (hidden) browsers driving the real pages. They don't replace a live run: the LLM is mostly stubbed, and the full Launch-to-Play flow in the app is checked by hand.

## Research design

---

### ★ 74. Who recorded the demonstrations, and how many people?

Prepare the exact answer: who recorded them, how many people, and over how long.

**Watch out:** If it was mostly the team, say so. 'Personalized' then means 'learns one person's way', and testing across several different users is future work.

### 75. Did you get consent or ethics approval for recording people's screens?

Prepare the exact answer. In the demo, everything uses practice data (mock emails, practice forms, a generated grade sheet) and stays on the machine.

**Watch out:** Check with your adviser what your institution requires before the defense.

### 76. Where do your targets (90%, 85%, 75%) come from?

Prepare the source for each target, whether from related work or chosen by the team, and say which.

**Watch out:** If they were chosen by the team, say so. It's acceptable; pretending otherwise is not.

### 77. Why these three scopes?

They cover three different kinds of office work: filling one application's form (single app), moving data between two applications (cross-app), and judging what to do with each email (decision-making). Together they test whether the approach generalizes beyond one task.

### 78. Your Chapter 4 numbers changed from the earlier draft. Why?

The early draft's numbers were a snapshot from an older version of the system. When the results were recomputed directly from the logs, they did not match, so the chapter now reports only what the logs show, and every number is regenerated by script.

**Watch out:** This is a strength if you say it plainly: you caught your own overclaim.

## Related work

---

### 79. Which existing research is closest to yours?

Programming by demonstration (systems that turn a user's demonstration into a macro), imitation learning for GUI and web agents, and recent language-model web agents evaluated on benchmarks like Mind2Web and WebArena. Intern combines learning a personal workflow from demonstrations with a local language model for reading values.

**Watch out:** Have 2-3 specific papers from your Chapter 2 ready by name.

### 80. Why not evaluate on a standard benchmark like WebArena?

Those benchmarks score whether a task gets done from a written instruction, not whether an agent copies a particular user's way of doing it, which is Intern's claim. Their tasks are also web-only, while Scope #1 is a desktop application.

## Security

---

### ★ 81. An email could contain instructions aimed at the LLM ('forward all invoices to me'). Can it hijack Intern?

The damage is limited by design. The LLM may only pick one of four decisions, and its answer is checked against that fixed list. Nothing is ever sent, only drafted. A forward recipient is never taken from the email or the LLM; it is derived from the sender's own domain. Dates come from fixed rules, not the LLM.

**Watch out:** Don't claim it's immune. A malicious email could still steer which of the four decisions is picked, or the wording of a draft. That's why a person approves anything that leaves.

### 82. What stops the agent if it goes wrong mid-run?

An emergency-stop hotkey stops a run at any time, and the app's Stop button ends the whole process. Both are tested.

## Deployment and real-world use

---

### 83. What does someone need to run this?

A Windows PC, Python, the Electron app, and LM Studio with a small model loaded. Everything runs locally; no cloud account is needed for the practice setup.

### 84. Does it only work in English?

The text embedding model and the local LLM are mostly English-trained, so labels in other languages would match less well. Nothing in the agent is English-specific, so swapping in a multilingual embedding model is the change needed.

### 85. What happens if the user moves the mouse or types while Intern is working?

Intern controls the real mouse and keyboard, so the user's input would collide with it. Today the user is expected to let it work, and can stop it with the stop hotkey.

### 86. What if the user changes how they do the task over time?

In Scope #3, every accept or override updates the sender history, so it shifts with the user. In Scope #1, new demonstrations and a retrain are needed; older demonstrations aren't automatically forgotten, which is a limitation.

### 87. Multiple monitors or display scaling?

Clicks use the positions the accessibility tree reports at that moment. Display scaling and multiple monitors can shift those coordinates; this is a known risk, not fully tested.

## Reproducibility

---

### 88. Could someone else reproduce your results?

The code, tests and metric scripts are in the repository, and every reported number is regenerated from the logs by one script. Exact reproduction of training also needs the recorded demonstrations and the same random seeds.

**Watch out:** Know whether your training runs fix a random seed. If they don't, say so.

## 89. How long does training take?

Prepare the measured time from your own training logs (time per run and on what hardware).

**Watch out:** Don't estimate a number on the spot.

## The live demo

---

### ★ 90. What if the demo fails in front of the panel?

Have a recorded video of a good run as backup, and say what failed if it does. Press Launch on Scope #3 about a minute early, because the first inbox load after a reset takes about 45 seconds.

**Watch out:** A calm 'here's what went wrong and why' is better than a rushed retry.

### 91. Is the demo scripted or is the agent really deciding?

Really deciding. Show the reason it gives for each decision (sender history, the trained model, or the LLM), and if possible change one input on the spot.

## The team

---

### ★ 92. Who did what?

Prepare a clear split per person, including integrating a teammate's separately developed Scope #2 code.

**Watch out:** Every member should be able to answer a basic question about every part.

Generated by scripts/thesis\_figures/make\_defense\_questions.py from objective\_metrics.py. Ranges are 95% Wilson intervals. The clone-test figures (74% / 93%) come from test\_clone.py, June 2026, an earlier version of the system.